



Prompt Engineering for Developers

Task

[Visit our website](#)

Introduction

Over the last several lessons, you have explored the foundations of Artificial Intelligence (AI), machine learning, large language models (LLMs), and the growing role of generative AI in modern web applications. You have also seen how AI can be integrated into applications, how user experiences must be designed around AI capabilities, and how conversational interfaces are becoming increasingly common in software products.

As AI becomes more deeply integrated into development workflows, a new skill has emerged that is rapidly becoming as valuable as searching documentation, debugging code, or reading technical specifications. That skill is prompt engineering.

Prompt engineering is the practice of designing effective instructions that guide an AI system toward producing useful, reliable, and consistent outputs. While AI models are capable of generating code, documentation, explanations, test cases, and design ideas, the quality of their output is directly influenced by the quality of the instructions they receive.

For developers, prompt engineering is not simply about asking better questions. It is about learning how to communicate requirements, provide context, establish constraints, and guide AI systems toward achieving specific technical objectives. Just as a poorly written software specification can lead to an unsuccessful project, a poorly written prompt can lead to inaccurate, incomplete, or unusable AI-generated output.

In this lesson, you will learn how modern AI systems interpret prompts, how to structure effective instructions, and how prompt engineering techniques can be applied to real-world development tasks such as coding, debugging, testing, documentation, and application design.

Understanding Prompt Engineering

At its simplest level, a prompt is any instruction or input provided to an AI system. When you interact with [ChatGPT](#), [GitHub Copilot](#), [Claude](#), [Hyperdev](#), or any other generative AI tool, every request you make is a prompt. Many people initially assume that prompt engineering is simply about asking questions. In reality, effective prompt engineering is much closer to writing technical requirements than having a casual conversation.

Consider the following prompt:

```
Create a login form.
```

Although this instruction appears straightforward, it leaves many important questions unanswered. Should the form use HTML only? Should JavaScript validation be included? Should accessibility considerations be addressed? Should the design be mobile-friendly? Should the output include CSS?

Because the instruction lacks detail, the AI system must make assumptions. Different models may make different assumptions, leading to inconsistent results. Now consider this alternative:

```
Create an accessible login form using HTML, CSS, and JavaScript.  
Include client-side validation, responsive styling for mobile  
devices, semantic HTML elements, and appropriate error messages.  
Return the complete code with explanatory comments.
```

This prompt provides significantly more guidance. The AI no longer needs to guess the developer's intentions because the requirements are clearly defined. Prompt engineering is the process of transforming vague instructions into clear, structured requests that help AI systems produce predictable and useful outcomes.

Why Prompt Engineering Matters for Developers

Throughout the history of software development, developers have relied on tools that help them work more efficiently. Integrated Development Environments (IDEs), version control systems, testing frameworks, package managers, and cloud platforms have all improved developer productivity. AI tools represent the next major evolution in developer tooling. Today, developers use AI systems to generate code, explain unfamiliar concepts, review existing implementations, create documentation, design user interfaces, generate test cases, and troubleshoot technical problems. These capabilities can save significant amounts of time when used effectively.

However, there is an important distinction between simply using AI and using AI effectively. Imagine two developers attempting to build the same feature. The first developer enters a vague prompt and receives a generic response that requires substantial editing and correction. The second developer carefully structures the prompt, specifies requirements, provides context, and defines the expected output format. Although both developers use the same AI tool, the second developer is far more likely to receive a useful result.

This difference is why prompt engineering has become such an important skill. Developers who understand how to communicate effectively with AI systems are often able to obtain better solutions more quickly while maintaining greater control over the development process.

Prompt engineering also helps developers reduce frustration. Many complaints about AI-generated outputs stem from poorly defined instructions rather than limitations of

the AI system itself. Learning how to write effective prompts allows developers to make better use of the capabilities that modern AI tools provide.

How Large Language Models Interpret Prompts

To understand why prompt engineering matters, it is useful to understand how large language models process information. Unlike traditional software applications, large language models do not follow a fixed set of predefined rules for every situation. Instead, they generate responses by identifying patterns in language and predicting the most appropriate output based on the information available.

This means the model relies heavily on the context supplied within the prompt. When a prompt contains little information, the model must fill in the gaps by making assumptions. Sometimes those assumptions are useful. Other times they lead to inaccurate or incomplete responses.

Suppose a developer submits the following prompt:

`Create a product page.`

A product page could refer to thousands of different implementations. The page could be intended for an e-commerce store, a software company, a mobile application, or a digital marketplace. It could be designed for desktop users, mobile users, or both. It may require accessibility support, search functionality, user reviews, or product recommendations. The AI system cannot know these requirements unless they are provided.

Now consider the following prompt:

`Create a responsive e-commerce product page using HTML, CSS, and JavaScript. Include a product image gallery, customer reviews section, add-to-cart button, accessibility-friendly labels, and mobile-responsive layouts. Explain the purpose of each major section.`

This version provides significantly more information. The AI system now has a clearer understanding of the intended outcome and can generate a more targeted response. The most effective prompts reduce uncertainty. The less guessing an AI system must perform, the more reliable the output typically becomes.

The Components of an Effective Prompt

Most successful prompts contain four fundamental elements: an objective, context, constraints, and an expected output format. Together, these elements provide the

information necessary for an AI system to understand both the task and the desired result.

Objective

The objective describes what you want the AI system to accomplish. This is the primary goal of the prompt and should be stated as clearly as possible. For example, a developer might want the AI to generate a React component, explain an error message, create documentation, or review existing code. The more specific the objective, the easier it becomes for the model to generate a useful response.

Consider the difference between these two instructions:

```
Help me with React.
```

and

```
Create a React component that displays a searchable list of products  
using local state management.
```

The second prompt establishes a clear objective, making it far easier for the AI system to respond effectively.

Context

Context provides background information about the situation. This information helps the AI understand who the audience is, what the project involves, and how the output will be used. Imagine asking an AI system to explain API authentication. The explanation would be very different depending on whether the audience consists of beginner developers, experienced software engineers, or business stakeholders. Providing context helps the model adapt its response to the specific situation.

For example:

```
Explain API authentication to beginner web development students who  
have recently learned about HTTP requests and JSON.
```

This additional information allows the AI system to produce a more appropriate explanation.

Constraints

Constraints define the boundaries within which the AI should operate. In software development, constraints are often just as important as the objective itself. A developer may need a solution that uses a specific framework, avoids external dependencies, follows accessibility standards, or remains within performance limitations. Without

constraints, an AI system may generate technically correct solutions that are unsuitable for the project's requirements.

Examples of constraints include:

- Use only JavaScript.
- Do not use external libraries.
- Follow accessibility best practices.
- Keep the solution under 50 lines of code.

Constraints help ensure that generated outputs align with project requirements and technical limitations.

Output Format

The final component of an effective prompt is the expected output format. AI systems can provide responses in many different forms, including code snippets, technical explanations, markdown documents, JSON objects, tables, or step-by-step instructions. By specifying the desired format, developers can reduce ambiguity and improve consistency.

For example:

```
Return the answer as a markdown table.
```

or

```
Provide the complete code solution followed by a brief explanation.
```

Clearly defining the format often improves the usability of the generated response.

Writing Structured Prompts

A useful way to think about prompt engineering is to view prompts as miniature technical specifications. When developers write software requirements, they define what must be built, who it is for, what limitations exist, and what outcomes are expected. Effective prompts follow a similar structure. A practical framework for prompt construction is:

Task + Context + Constraints + Output Format

This framework encourages developers to include the most important information needed by the AI system. Consider the following example:

```
Create a responsive navigation bar using HTML, CSS, and JavaScript.  
The navigation should support mobile devices and include a hamburger
```

menu. Do not use Bootstrap or any external libraries. Return complete code with comments explaining each section.

The task is clearly defined. The context identifies the intended feature. The constraints specify technology limitations. The output format explains how the response should be structured. This approach reduces ambiguity and typically produces more useful results than short, open-ended instructions.

As you continue through this lesson, you will see how this framework can be expanded and adapted for a variety of development activities, including code generation, debugging, testing, documentation, and AI-assisted software design.

Prompt Engineering Techniques

In the previous section, you learned that the quality of an AI system's output is heavily influenced by the quality of the prompt it receives. You also explored the key components of effective prompts and saw how developers can structure instructions to reduce ambiguity and improve consistency.

While a well-structured prompt provides a strong foundation, experienced developers often use additional prompt engineering techniques to guide AI systems more effectively. These techniques help improve accuracy, establish consistent formatting, and produce outputs that better match the developer's requirements.

In this section, we will explore four of the most widely used prompt engineering approaches: zero-shot prompting, one-shot prompting, few-shot prompting, and role-based prompting.

Zero-Shot Prompting

Zero-shot prompting is the simplest and most common prompting technique. In a zero-shot prompt, the AI system is given a task without any examples of the desired output. The model must rely entirely on its existing knowledge and the instructions contained within the prompt.

For example, consider the following prompt:

```
Create a JavaScript function that validates whether an email address follows a valid format.
```

The prompt clearly states the task but provides no examples. The AI system must generate a solution based solely on its understanding of JavaScript and email validation patterns. Zero-shot prompting is often effective when requesting straightforward tasks,

such as generating simple code snippets, explaining programming concepts, or creating basic documentation.

For example, a developer might ask:

```
Explain the difference between let, const, and var in JavaScript.
```

or

```
Create a CSS grid layout with three equal-width columns.
```

In both cases, the task is relatively clear, and the model can usually generate useful responses without requiring additional examples. However, zero-shot prompting has limitations. When a task involves a specific format, style, or business requirement, the AI system may interpret the request differently than intended. This can lead to inconsistencies, especially when multiple developers are working within the same project. As projects become more complex, developers often move beyond zero-shot prompting and begin providing examples to guide the model's behaviour.

One-Shot Prompting

One-shot prompting introduces a single example before asking the AI system to complete a similar task. The example serves as a reference point that helps the model understand the expected structure, style, or format of the output. Imagine that a development team maintains documentation for user interface components using a standard format. The team provides the following example:

Component: Button

Purpose: Allows users to trigger actions within the interface.

Props: label, onClick

Example Usage: `<Button label="Save" />`

The developer then asks:

```
Using the same format, document a React Card component.
```

Because the AI system has already seen an example, it can more easily replicate the structure in its response. One-shot prompting is particularly useful when consistency matters. Documentation, test cases, API descriptions, and user interface specifications often benefit from this approach because the example acts as a template.

Developers frequently use one-shot prompting when introducing a new coding standard, documentation format, or naming convention to an AI system. The technique

is simple but often produces significantly better results than relying on zero-shot prompting alone.

Few-Shot Prompting

Few-shot prompting expands on the previous technique by providing multiple examples before requesting a new output. Rather than showing the model a single example, the developer supplies several examples that demonstrate the desired pattern. This approach is especially useful when consistency and accuracy are critical. Imagine a development team creating API documentation. Each endpoint follows a particular structure:

- **Example 1**

Endpoint: GET /users

Purpose: Retrieve all users.

Response: List of user objects.

- **Example 2**

Endpoint: POST /users

Purpose: Create a new user.

Response: Newly created user object.

The developer then asks:

Create documentation for DELETE /users/{id} using the same format.

Because the model has been shown multiple examples, it has a stronger understanding of the pattern it should follow. Few-shot prompting is widely used in professional AI workflows because it reduces variation in outputs. Instead of leaving the AI system to interpret the desired format independently, the developer explicitly demonstrates the expected structure.

This technique is particularly valuable when generating:

- Documentation
- Code comments
- User interface content
- API descriptions
- Test cases
- Structured data outputs

The more clearly the examples reflect the desired outcome, the more likely the AI system is to produce consistent results.

Role-Based Prompting

One of the most powerful prompt engineering techniques is role-based prompting. Role-based prompting involves assigning a specific role, perspective, or area of expertise to the AI system before giving it a task. This technique helps influence how the model approaches the problem and shapes the style of the response.

For example:

```
Act as a senior front-end developer and review the following React component.
```

or

```
Act as an accessibility specialist and evaluate this HTML form.
```

or

```
Act as a software architect and recommend improvements to this system design.
```

These instructions encourage the AI system to focus on specific considerations that align with the assigned role. Consider the difference between the following prompts:

```
Explain form validation.
```

and

```
Act as a senior web development instructor and explain form validation to beginner developers using practical examples.
```

The second prompt provides considerably more direction. It establishes both expertise and audience, helping the AI system produce a more focused response. Role-based prompting is particularly useful when developers need assistance from different perspectives during the software development lifecycle.

For example:

- A front-end developer may focus on usability and accessibility.
- A software architect may focus on scalability and maintainability.
- A security specialist may focus on vulnerabilities and attack surfaces.
- A technical writer may focus on clarity and documentation quality.

By assigning these roles, developers can encourage AI systems to evaluate problems through different lenses.

Combining Prompt Engineering Techniques

In practice, developers rarely use these techniques in isolation. Many of the most effective prompts combine multiple approaches to provide the AI system with as much guidance as possible. Consider the following example:

`Act as a senior React developer.`

`Review the following components and identify opportunities to improve accessibility.`

`Use the same review format shown below.`

`Example:`

`Issue: Missing form labels`

`Impact: Screen readers cannot identify inputs correctly.`

`Recommendation: Add associated label elements.`

`Component:`

`[Insert Component Code]`

This prompt combines role-based prompting with one-shot prompting. The assigned role establishes expertise, while the example demonstrates the desired review format. Combining techniques in this way often produces more reliable and predictable outputs than using a single prompting strategy. As developers gain experience working with AI systems, they naturally begin blending these techniques based on the requirements of each task.

Choosing the Right Technique

Different prompting techniques are suited to different situations. Zero-shot prompting works well when requesting simple explanations, basic code snippets, or straightforward tasks. One-shot prompting is useful when consistency is important and a single example can establish the desired format.

Few-shot prompting is often the best choice when strict formatting, repeatability, or structured outputs are required. Role-based prompting is valuable whenever a developer wants the AI system to approach a problem from a particular perspective or level of expertise. The choice of technique depends on the complexity of the task, the importance of consistency, and the amount of guidance required. Experienced

developers learn to recognize when additional context, examples, or role assignments are needed and adjust their prompts accordingly.

Prompt Engineering in Everyday Development Work

Prompt engineering techniques are no longer limited to research environments or AI specialists. They are increasingly becoming part of everyday software development. A developer using an AI-powered coding assistant may unknowingly use role-based prompting through comments and instructions. A team generating documentation may rely on few-shot prompting to maintain consistency across dozens of API endpoints. A student learning React may use zero-shot prompting to receive explanations and examples for unfamiliar concepts.

As AI tools become more deeply integrated into development environments, understanding these techniques becomes increasingly valuable. Developers who know how to guide AI systems effectively are often able to work more efficiently, generate higher-quality outputs, and maintain greater control over the development process.

Prompt Engineering for Development Tasks

Understanding prompting techniques is valuable, but the real benefit of prompt engineering becomes clear when these techniques are applied to practical development work. Modern developers increasingly use AI systems throughout the software development lifecycle, from generating initial code and debugging issues to creating documentation and designing user experiences.

The quality of the results obtained from AI tools often depends less on the tool itself and more on how effectively the developer communicates the task. Prompt engineering allows developers to provide clearer instructions, establish expectations, and guide AI systems toward producing outputs that align with project requirements.

In this section, we will explore how prompt engineering can be applied to common development activities and how developers can improve results through iterative refinement.

Prompt Engineering for Code Generation

Code generation is one of the most common uses of AI within software development. Modern AI systems can generate functions, components, layouts, APIs, and complete application features. However, simply asking an AI tool to "write some code" often leads to generic results that require significant modification.

Consider the following prompt:

```
Create a web page.
```

While technically valid, this instruction provides very little information. The AI system has no understanding of the purpose of the page, the technologies involved, the intended audience, or the required functionality.

A more effective prompt might be:

```
Create a responsive landing page for an online learning platform using HTML, CSS, and JavaScript. Include a hero section, course highlights, student testimonials, and a contact form. Follow accessibility best practices and provide comments explaining the structure of the code.
```

The second prompt establishes a clear objective, provides context, includes constraints, and specifies expectations for the output. As a result, the generated solution is likely to be more useful and closer to the developer's needs. When generating code with AI, developers should provide information about:

- The technologies being used.
- The purpose of the feature.
- Required functionality.
- Accessibility requirements.
- Performance considerations.
- Desired output format.

The more relevant information provided, the more accurately the AI can respond.

Prompt Engineering for Debugging

Debugging is another area where AI systems can provide significant value. Developers frequently use AI tools to investigate errors, understand unexpected behaviour, and identify potential solutions. However, debugging prompts often fail because insufficient information is provided. Consider this prompt:

```
My code is broken. Fix it.
```

Although the AI may attempt to help, the instruction does not explain what the code is supposed to do, what error is occurring, or where the problem might be located. A more effective approach would be:

The following JavaScript code throws a `TypeError` when a user submits the form. Explain what causes the error, identify the problematic line, and provide a corrected solution. Also explain how the fix works.

This prompt provides both context and expectations. The AI now understands the task and can focus on identifying the root cause of the issue. When requesting debugging assistance, developers should include:

- The relevant code.
- Error messages.
- Expected behaviour.
- Actual behaviour.
- Recent changes that may have introduced the issue.

Providing this information allows the AI system to produce more accurate recommendations and explanations.

Prompt Engineering for Documentation

Documentation is an essential part of software development, yet it is often neglected because it can be time-consuming to produce. AI systems can assist by generating technical documentation, API descriptions, setup guides, and user instructions. The quality of documentation generated by AI depends heavily on the clarity of the prompt. For example:

Create documentation for this API.

This instruction leaves many questions unanswered. Should the documentation include example requests? Error responses? Authentication requirements? Usage examples?

A more structured prompt might be:

Create developer documentation for the following REST API endpoint. Include a description of its purpose, request parameters, authentication requirements, example requests, example responses, and possible error codes. Write for junior developers.

This prompt provides both structure and audience information, helping the AI generate more useful documentation.

Documentation prompts often benefit from one-shot or few-shot prompting because examples help establish consistent formatting across multiple documents.

Prompt Engineering for Test Generation

Testing plays a critical role in software quality, and AI systems can assist developers by generating test scenarios and test cases. Suppose a developer wants to test a user registration form. A basic prompt might be:

```
Generate tests for a registration form.
```

While this may produce useful results, the AI may overlook important edge cases. A more detailed prompt could be:

```
Generate ten test cases for a user registration form. Include positive test cases, negative test cases, edge cases, validation scenarios, and expected outcomes. Assume the form contains fields for name, email address, password, and password confirmation.
```

This prompt encourages broader test coverage and provides a clearer understanding of the requirements. Developers often use AI-generated tests as a starting point and then refine them based on project-specific requirements.

Managing Hallucinations and Inaccuracies

One of the most important responsibilities when working with AI systems is understanding their limitations. Although modern AI models can generate highly convincing responses, they do not guarantee correctness. Occasionally, they produce information that appears accurate but is actually incorrect. This phenomenon is commonly referred to as hallucination. For developers, hallucinations may appear in several forms. An AI system might invent a function that does not exist, generate incorrect API documentation, suggest outdated syntax, or provide code that contains subtle bugs.

For example, an AI system may confidently recommend a library function that was never part of the library or provide implementation details that differ from official documentation.

This is why AI-generated output should always be treated as a starting point rather than a final solution. Developers should:

- Test generated code before deployment.

- Verify technical information against official documentation.
- Review security-related recommendations carefully.
- Confirm that suggested APIs and libraries actually exist.
- Validate business logic before implementation.

Prompt engineering can reduce the likelihood of hallucinations by encouraging the AI to focus on specific requirements and trusted sources of information, but it cannot eliminate inaccuracies entirely. Human oversight remains essential.

Iterative Prompt Refinement

One of the biggest misconceptions about prompt engineering is that the perfect prompt must be written on the first attempt.

In reality, effective prompting is often an iterative process.

Developers rarely receive the exact output they need from the very first prompt. Instead, they gradually improve the prompt based on the responses they receive.

A typical workflow may look like this:

1. Write an initial prompt.
2. Review the generated output.
3. Identify missing information or weaknesses.
4. Add additional context or constraints.
5. Refine the instructions.
6. Generate a revised response.

This process closely resembles software development itself. Developers write code, test the results, identify issues, and improve the solution through multiple iterations. Each refinement moves the AI closer to the desired outcome. Prompt engineering is therefore less about writing perfect prompts and more about learning how to improve prompts systematically.

Prompt Templates for Developers

As developers gain experience with AI systems, many create reusable prompt templates that can be adapted for different tasks. A code-generation template might follow this structure:

Act as a senior web developer.

Create [feature].

Use [technology stack].

Requirements: [requirements].

Constraints: [limitations].

Return complete code with explanatory comments.

A debugging template might be structured as follows:

Act as a senior software engineer.

Review the following code and error message.

Explain the root cause, identify the affected lines, and provide a corrected solution.

Code:

[Insert code here]

A documentation template could include:

Create technical documentation for the following component.

Include:

Purpose

Inputs

Outputs

Example usage

Limitations

Templates help developers maintain consistency while reducing the effort required to construct prompts from scratch.

Prompt Engineering Tools for Developers

As AI adoption continues to grow, developers now have access to a wide range of tools that incorporate prompt engineering into their workflows. Although the underlying principles remain the same, each platform offers different strengths and capabilities. Understanding how these tools are used within the industry can help you develop prompt engineering skills that transfer across multiple environments.

ChatGPT

ChatGPT is one of the most widely used AI tools in software development. Developers use it to generate code, explain technical concepts, review implementations, create documentation, and investigate errors. Its conversational interface makes it particularly useful for iterative prompt refinement because developers can continuously improve their instructions and build upon previous responses.

GitHub Copilot

GitHub Copilot integrates directly into development environments such as VS Code and provides real-time code suggestions while developers work. Although Copilot often appears to generate code automatically, prompt engineering still plays an important role. Function names, comments, documentation, and surrounding code all act as prompts that influence the suggestions generated by the tool.

Cursor

Cursor is an AI-first development environment that combines traditional coding tools with conversational AI capabilities. Developers can ask questions about entire codebases, generate features, refactor applications, and receive explanations directly within the editor. Because Cursor can access broader project context, clear instructions become even more important when working on larger applications.

Claude

Claude is frequently used for architectural discussions, documentation creation, code reviews, and reasoning-intensive tasks. Many developers use Claude when they need detailed explanations, long-form content, or assistance working with large amounts of information. Its ability to maintain context across lengthy conversations makes it useful for more complex development activities.

Hyperdev

Hyperdev provides access to AI-powered development assistance designed to support learning, experimentation, and software development workflows. As you continue through this bootcamp, you are encouraged to use Hyperdev to practice the prompt engineering techniques introduced in this lesson. Experiment with different prompt structures, compare the outputs generated by different instructions, and observe how additional context, constraints, and examples influence the quality of responses.

Before moving on to the practical task, spend some time exploring Hyperdev and testing several prompts related to concepts you have covered throughout this bootcamp. Try generating code, requesting explanations, creating documentation, and

debugging simple examples. You can access Hyperdev through your student dashboard.

The goal is not simply to use the tool but to actively experiment with prompt engineering principles and observe how they affect the quality and usefulness of AI-generated outputs.

Real-World Applications in Web Development

Prompt engineering is rapidly becoming part of everyday development practice. Developers use AI systems to accelerate development, improve productivity, and support problem-solving across a wide variety of tasks.

Modern web development teams increasingly rely on prompt engineering to generate user interface components, create API integrations, produce documentation, generate test cases, review code, investigate bugs, and assist with software design decisions.

As AI capabilities continue to evolve, the ability to communicate effectively with these systems will become an increasingly valuable professional skill. Developers who understand how to structure prompts, provide context, define constraints, and refine instructions are often able to obtain more accurate and useful results while maintaining greater control over the development process.

Instructions

You have joined a development team that is building a web application called **SkillTrack**, a platform that helps learners track their progress through online courses. The development team wants to use AI tools to assist with several common software development activities, including code generation, debugging, documentation, and testing.

Your responsibility is to design prompts that help the AI produce useful outputs while ensuring that the prompts are clear, structured, and appropriate for the task.

Create a website named **prompt_engineering_portfolio** that documents your prompt engineering process. Your website must contain the following sections. Your submission must:

- Be developed using HTML and CSS.
- Be structured as a professional web page.
- Include clear section headings.
- Use appropriate spacing and visual hierarchy.
- Be readable on desktop screens.
- Demonstrate professional presentation standards.

You may use JavaScript if desired, but it is not required.



Practical task

Section 1 – Prompt Analysis

Explain, in your own words:

- What prompt engineering is.
- Why prompt engineering is important for developers.
- How poorly written prompts can impact AI-generated outputs.
- Why developers should refine prompts iteratively.

This section should be written as professional content rather than short answers.

Section 2 – Code Generation Prompt

Create an initial prompt that asks an AI system to generate a feature for the SkillTrack platform. Examples include:

- Student dashboard
- Progress tracker
- Course search feature
- User profile page
- Course recommendation panel

Display:

1. Your original prompt.
2. A short explanation of why the prompt is weak or incomplete.
3. An improved version of the prompt.
4. A comparison explaining how the improved prompt provides:
 - Better context
 - Better constraints
 - Better output requirements

You do not need to include the AI-generated response. The focus is on prompt design.

Section 3 – Debugging Prompt

You are provided with the following buggy JavaScript code:

```
const students = [
  { name: "Sarah", completed: 5 },
  { name: "John", completed: 8 },
  { name: "Aisha", completed: 12 }
];

function calculateAverageCompletion(data) {
  let total = 0;
  for (let i = 0; i <= data.length; i++) {
    total += data[i].completed;
  }
  return total / data.length;
}

console.log(calculateAverageCompletion(students));
```

Create:

1. A poor debugging prompt.
2. An improved debugging prompt.
3. A written explanation of why the improved prompt would produce better debugging assistance.

Your explanation should reference concepts from the lesson.

Section 4 – Documentation Prompt

The SkillTrack application contains a feature that allows users to search for courses by keyword. Design a prompt that would instruct an AI system to generate professional developer documentation for this feature.

Your prompt should clearly define:

- Audience
- Context
- Required sections
- Formatting requirements

After presenting the prompt, explain why each component was included.

Section 5 – Testing Prompt

Design a prompt that asks an AI system to generate test cases for a user registration form.

The form contains:

- Full Name

- Email Address
- Password
- Confirm Password

Your prompt must request:

- Positive test cases
- Negative test cases
- Edge cases
- Expected outcomes

Provide a short explanation describing how your prompt improves test coverage.

Section 6 – Prompt Refinement Challenge

This section demonstrates your ability to improve prompts iteratively. Start with the following prompt:

Build a dashboard.

Improve the prompt through three separate revisions. For each revision:

- Show the revised prompt.
- Explain what was improved.
- Explain why the change should lead to a better output.

Your final version should demonstrate clear objectives, context, constraints, and output requirements.

Section 7 – AI Tool Comparison

Use at least one AI tool discussed in the lesson. Examples include:

- ChatGPT
- GitHub Copilot
- Cursor
- Claude
- Hyperdev

Explain:

- Which tool you selected.
- Why you selected it.
- One strength of the tool.
- One limitation of the tool.
- How prompt engineering affects the quality of results generated by that tool.

You are encouraged to experiment with Hyperdev before completing this section.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
